

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет Программной Инженерии и Компьютерной Техники

Системы компьютерной обработки изображений

Лабораторная работа № 1

«Вычисление освещенности в точках поверхности»

Выполнил студент

Стрельбицкий Илья Павлович

Группа № Р3413

Преподаватель: Жданов Дмитрий Дмитриевич

г. Санкт-Петербург 2025

Цель

Изучить, как вычисляется освещенность в точках точки на плоскости треугольника при освещении его точечным источником света с заданной диаграммой излучения.

Используемые формулы

1. «Цветная» интенсивность источника под углом к оси источника света:

$I(RGB, \vec{s}) = I_0(RGB) \cos \theta$, где $I_0(RGB)$ – «цветная» интенсивность источника света в направлении его оси $\vec{O}$, $\|\vec{O}\| = 1$, θ - угол между направлением распространения света и осью источника света, $\cos \theta$ – диаграмма излучения.

2. «Цветная» освещенность точки:

$E(RGB, \vec{P}_T) = \frac{I(RGB, \vec{s}) \cos \alpha}{R^2}$, где $I(RGB, \vec{s})$ – «цветная» интенсивность света, α - угол между направлением света и нормалью к освещаемой поверхности, R - расстояние от источника света до рассматриваемой точки.

3. Перевод локальных координат точки в плоскости в глобальные

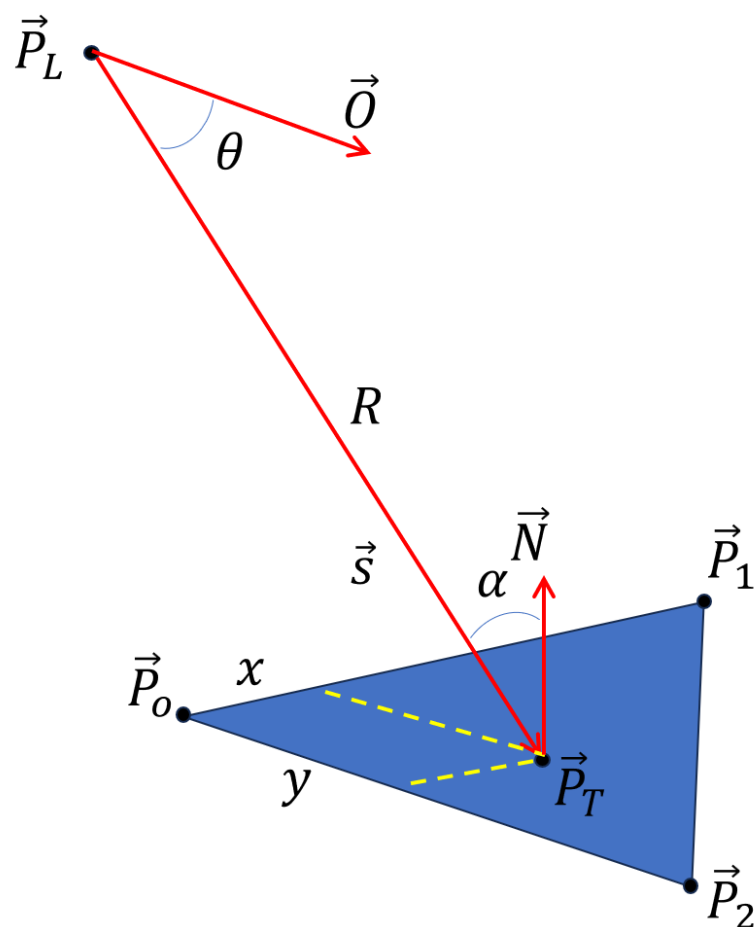
$\vec{P}_T = \vec{P}_0 + \left(\frac{\vec{P}_1 - \vec{P}_0}{\|\vec{P}_1 - \vec{P}_0\|} \cdot x + \frac{\vec{P}_2 - \vec{P}_0}{\|\vec{P}_2 - \vec{P}_0\|} \cdot y \right)$, где x и y смещения по ребрам треугольника.

4. Вычисление вектора нормали плоскости треугольника через 3 точки:

$$\vec{N} = \frac{(\vec{P}_2 - \vec{P}_0) \times (\vec{P}_1 - \vec{P}_0)}{\|(\vec{P}_1 - \vec{P}_0) \times (\vec{P}_2 - \vec{P}_0)\|}$$

5. Вектор от точки плоскости до источника света:

$$\vec{s} = \vec{P}_T - \vec{P}_L, R^2 = \|\vec{s}\|^2, \cos \alpha = \frac{\vec{s} \cdot \vec{N}}{\|\vec{s}\|}, \cos \theta = \frac{\vec{s} \cdot \vec{O}}{\|\vec{s}\|}$$



Шаги алгоритма

1. Инициализация параметров:

- Задать интенсивность источника света $I_0(RGB)$
- Задать вектор направления источника $\vec{O}$
- Задать координаты источника света $\vec{P}_L$
- Задать вершины треугольника $\vec{P}_0, \vec{P}_1, \vec{P}_2$
- Задать локальные координаты точек (x, y) , для которых будем проводить дальнейшие расчеты

2. Вычисление нормали плоскости:

- Рассчитать векторы ребер:

$$\vec{v}_1 = \vec{P}_1 - \vec{P}_0$$

$$\vec{v}_2 = \vec{P}_2 - \vec{P}_0$$

b. Вычислить векторное произведение:

$$\vec{N} = \vec{v}_1 \times \vec{v}_2$$

c. Выполнить нормализацию полученной нормали:

$$\vec{N} = \frac{\vec{N}}{|\vec{N}|}$$

3. Выполнение расчетов для каждой точки (x, y):

a. Перевод в глобальные координаты:

$$\vec{P}_T = \vec{P}_0 + \frac{\vec{v}_1}{|\vec{v}_1|} * x + \frac{\vec{v}_2}{|\vec{v}_2|} * y$$

b. Вектор от точки к источнику света:

$$\vec{s} = \vec{P}_L - \vec{P}_T; R = |\vec{s}|$$

c. Расчет интенсивности:

$$\cos \theta = \frac{\vec{s} \cdot \vec{O}}{R}; I(RGB) = I_0(RGB) * |\cos \theta|$$

d. Расчет освещенности:

$$\cos \alpha = \frac{\vec{s} \cdot \vec{N}}{R}; E(RGB) = \frac{I(RGB) * |\cos \alpha|}{R^2}$$

Программа для расчета освещенности

Используется язык программирования Python3.

Для упрощения расчетов используется библиотека numpy

```
import numpy as np
```

```
def calculate_illumination(
    I0_RGB: tuple[float, float, float],
    vect_O: tuple[float, float, float],
    vect_P_L: tuple[float, float, float],
    vect_P0: tuple[float, float, float],
    vect_P1: tuple[float, float, float],
    vect_P2: tuple[float, float, float],
    x_values: tuple[float],
    y_values: tuple[float]
) -> dict[tuple[float, float], tuple[float, float, float]]:
    """
    Расчет освещенности точек на плоскости треугольника
    Параметры:
    I0_RGB - кортеж (R, G, B) интенсивности источника света
    vect_O - кортеж (x, y, z) направления оси источника (единичный вектор)
    vect_P_L - кортеж (x, y, z) положения источника света
```

```

vect_P0, vect_P1, vect_P2 - кортежи вершин треугольника
x_values - список значений x координат [x1, x2, x3, x4, x5]
y_values - список значений y координат [y1, y2, y3, y4, y5]
Результат:
results - словарь полученных значений в формате (x,y): (E_RGB_x, E_RGB_y,
E_RGB_z)
"""
# Преобразование в numpy массивы
I0: np.ndarray = np.array(I0_RGB, dtype=float)
O: np.ndarray = np.array(vect_O, dtype=float)
P_L: np.ndarray = np.array(vect_P_L, dtype=float)
P0: np.ndarray = np.array(vect_P0, dtype=float)
P1: np.ndarray = np.array(vect_P1, dtype=float)
P2: np.ndarray = np.array(vect_P2, dtype=float)
# Проверка: vect(O) должен быть единичным
O_norm: np.float64 = np.linalg.norm(O)
if abs(O_norm - 1.0) > 1e-10:
    O = O / O_norm
    print(f"Предупреждение: vect(O) нормализован до единичной длины")
# Вычисление нормали плоскости
v1: np.ndarray = P1 - P0
v2: np.ndarray = P2 - P0
N: np.ndarray = np.cross(v1, v2) # Внешняя нормаль
N_norm: np.float64 = np.linalg.norm(N)
if N_norm == 0:
    raise ValueError("Точки треугольника коллинеарны, нормаль не
определена")
N = N / N_norm # Нормализация нормали
# Единичные векторы ребер для локальной системы координат
u1: np.ndarray = v1 / np.linalg.norm(v1)
u2: np.ndarray = v2 / np.linalg.norm(v2)
# генерируем список всех точек
points: list[tuple[float, float]] = [(x, y) for x in x_values for y in
y_values]
# Начинаем вычислять значения освещенности для каждой точки
results: dict[tuple[float, float], tuple[float, float, float]] = {}
for x, y in points:
    try:
        # 1. Перевод локальных координат в глобальные
        P_T: np.ndarray = P0 + u1 * x + u2 * y
        # 2. Вектор от точки к источнику света и расстояние
        s: np.ndarray = P_L - P_T
        R: np.float64 = np.linalg.norm(s)
        if R < 1e-10: # Избегаем деления на ноль
            results[(x, y)] = (0.0, 0.0, 0.0)
            continue
        # 3. Расчет интенсивности I(RGB, vect(s))
        cos_theta: float = np.dot(s, O) / R
        I_RGB: np.ndarray = I0 * abs(cos_theta)
        # 4. Расчет освещенности E(RGB, vect(P_T))
        cos_alpha: float = np.dot(s, N) / R
        E_RGB: np.ndarray = I_RGB * abs(cos_alpha) / (R * R)
        E_RGB_x, E_RGB_y, E_RGB_z = E_RGB
        results[(x, y)] = (E_RGB_x, E_RGB_y, E_RGB_z)
    except Exception as e:

```

```

        print(f"Ошибка при расчете точки ({x}, {y}): {e}")
        results[(x, y)] = (0.0, 0.0, 0.0)
    return results

def print_results_table(
    results: dict[tuple[float, float], tuple[float, float, float]],
    x_values: tuple[float],
    y_values: tuple[float]
) -> None:
    """Красивое отображение результатов в виде таблицы"""
    print("Освещенность E(RGB, vect(P)) для точек треугольника:")
    # Определяем ширину столбцов
    num_cols = len(x_values)
    col_width = 17
    # Верхняя граница таблицы
    top_border = "+" + "-" * 6 + "+" + ("-" * (col_width + 4) + "+") * num_cols
    print(top_border)
    # Заголовок с x координатами
    header = "| y\\x |"
    for x in x_values:
        header += f" {x:^{col_width + 2}.2f} |"
    print(header)
    print(top_border)
    # Отображение данных
    for y in y_values:
        row = f"| {y:3.2f} |"
        for x in x_values:
            E = results.get((x, y), (0, 0, 0))
            # Форматируем RGB значения
            rgb_str = f"{E[0]:.3f}, {E[1]:.3f}, {E[2]:.3f}"
            row += f" {rgb_str:^{col_width - 1}} |"
        print(row)
    print(top_border)

def main() -> None:
    # Входные данные
    I0_RGB = (50.0, 20.8, 15.6) # I0(RGB)
    vect_O = (0, -0.2, 1) # Ось источника
    vect_P_L = (0, 0, 5) # Положение источника
    vect_P0 = (0, 0, 0) # Вершины треугольника
    vect_P1 = (2, 0, 0)
    vect_P2 = (0, 2, 0)
    # Значения x1-x5 и y1-y5
    x_1, x_2, x_3, x_4, x_5 = 0.0, 0.75, 1.5, 2.25, 3.0
    y_1, y_2, y_3, y_4, y_5 = 0.0, 0.3, 0.5, 1.0, 1.5
    x_values: tuple = (x_1, x_2, x_3, x_4, x_5)
    y_values: tuple = (y_1, y_2, y_3, y_4, y_5)
    # Вывод входных параметров
    print("Входные параметры:")
    print(f"x1-x5: {x_values}")
    print(f"y1-y5: {y_values}")
    print(f"I0(RGB): {I0_RGB}")
    print(f"vect(O): {vect_O}")
    print(f"vect(P_L): {vect_P_L}")
    print(f"Треугольник: P0= {vect_P0}, P1= {vect_P1}, P2= {vect_P2}")

```

```

print()
# Расчет освещенности
results = calculate_illumination(I0_RGB, vect_O, vect_P_L, vect_P0, vect_P1,
vect_P2, x_values, y_values)
# Вывод результатов
print_results_table(results, x_values, y_values)
print(f"\nДополнительная информация для проверки расчетов:")
v1 = np.array(vect_P1) - np.array(vect_P0)
v2 = np.array(vect_P2) - np.array(vect_P0)
N = np.cross(v1, v2)
N_norm = N / np.linalg.norm(N)
print(f"Нормаль плоскости: ({N_norm[0]:.3f}, {N_norm[1]:.3f},
{N_norm[2]:.3f})")

if __name__ == "__main__":
    main()

```

Вывод программы в консоль:

```

Входные параметры:
x1-x3: (0.0, 0.75, 1.5, 2.25, 3.0)
y1-y3: (0.0, 0.3, 0.5, 1.0, 1.5)
I0(RGB): (50.0, 20.8, 15.6)
vect(O): (0, -0.2, 1)
vect(P_L): (0, 0, 5)
Треугольник: P0=(0, 0, 0), P1=(2, 0, 0), P2=(0, 2, 0)

Предупреждение: vect(O) нормализован до единичной длины
Освещенность E(RGB, vect(P)) для точек треугольника:

```

y\x	0.00	0.75	1.50	2.25	3.00
0.00	1.961, 0.816, 0.612	1.876, 0.780, 0.585	1.651, 0.687, 0.515	1.356, 0.564, 0.423	1.060, 0.441, 0.331
0.30	1.970, 0.820, 0.615	1.885, 0.784, 0.588	1.659, 0.690, 0.518	1.364, 0.568, 0.426	1.067, 0.444, 0.333
0.50	1.961, 0.816, 0.612	1.876, 0.781, 0.585	1.653, 0.688, 0.516	1.361, 0.566, 0.425	1.066, 0.443, 0.333
1.00	1.886, 0.784, 0.588	1.807, 0.752, 0.564	1.597, 0.664, 0.498	1.321, 0.550, 0.412	1.041, 0.433, 0.325
1.50	1.750, 0.728, 0.546	1.680, 0.699, 0.524	1.493, 0.621, 0.466	1.244, 0.518, 0.388	0.989, 0.411, 0.308

```

Треугольник: P0=(0, 0, 0), P1=(2, 0, 0), P2=(0, 2, 0)

Предупреждение: vect(0) нормализован до единичной длины
Освещенность E(RGB, vect(P)) для точек треугольника:
+-----+-----+-----+-----+-----+
| y\x | | 0.00 | | 0.75 | | 1.50 | | 2.25 | | 3.00 | |
+-----+-----+-----+-----+-----+
| 0.00 | 1.961, 0.816, 0.612 | 1.876, 0.780, 0.585 | 1.651, 0.687, 0.515 | 1.356, 0.564, 0.423 | 1.060, 0.441, 0.331 |
+-----+-----+-----+-----+-----+
| 0.30 | 1.970, 0.820, 0.615 | 1.885, 0.784, 0.588 | 1.659, 0.690, 0.518 | 1.364, 0.568, 0.426 | 1.067, 0.444, 0.333 |
+-----+-----+-----+-----+-----+
| 0.50 | 1.961, 0.816, 0.612 | 1.876, 0.781, 0.585 | 1.653, 0.688, 0.516 | 1.361, 0.566, 0.425 | 1.066, 0.443, 0.333 |
+-----+-----+-----+-----+-----+
| 1.00 | 1.886, 0.784, 0.588 | 1.807, 0.752, 0.564 | 1.597, 0.664, 0.498 | 1.321, 0.550, 0.412 | 1.041, 0.433, 0.325 |
+-----+-----+-----+-----+-----+
| 1.50 | 1.750, 0.728, 0.546 | 1.680, 0.699, 0.524 | 1.493, 0.621, 0.466 | 1.244, 0.518, 0.388 | 0.989, 0.411, 0.308 |
+-----+-----+-----+-----+-----+

Дополнительная информация для проверки расчетов:
Нормаль плоскости: (0.000, 0.000, 1.000)

Process finished with exit code 0

```

Пример входных данных и результаты

1. $I_0(RGB) = (50.0, 20.8, 15.6)$

2. $\vec{O} = (0, -0.2, 1)$

3. $\vec{P}_L = (0, 0, 5)$

4. $\vec{P}_0 = (0, 0, 0)$

5. $\vec{P}_1 = (2, 0, 0)$

6. $\vec{P}_2 = (0, 2, 0)$

7. $x_1 = 0.00, y_1 = 0.00$

$$x_2 = 0.75, y_2 = 0.30$$

$$x_3 = 1.50, y_3 = 0.50$$

$$x_4 = 2.25, y_4 = 1.00$$

$$x_5 = 3.00, y_5 = 1.50$$

Вычисленные значения освещенности $E(RGB, \vec{P}_T)$ для точек, заданных локальными координатами:

$x \backslash y$	0.00	0.75	1.50	2.25	3.00
0.00	1.961, 0.816, 0.612	1.876, 0.780, 0.585	1.651, 0.687, 0.515	1.356, 0.564, 0.423	1.060, 0.441, 0.331
0.30	1.970, 0.820, 0.615	1.885, 0.784, 0.588	1.659, 0.690, 0.518	1.364, 0.568, 0.426	1.067, 0.444, 0.333
0.50	1.961, 0.816, 0.612	1.876, 0.781, 0.585	1.653, 0.688, 0.516	1.361, 0.566, 0.425	1.066, 0.443, 0.333
1.00	1.886, 0.784, 0.588	1.807, 0.752, 0.564	1.597, 0.664, 0.498	1.321, 0.550, 0.412	1.041, 0.433, 0.325
1.50	1.750, 0.728, 0.546	1.680, 0.699, 0.524	1.493, 0.621, 0.466	1.244, 0.518, 0.388	0.989, 0.411, 0.308

Вывод

В результате выполнения работы был изучен и реализован алгоритм вычисления освещенности точки на плоскости от источника света. Рассмотрены основные этапы вычислений, включая преобразование координат, определение нормали к плоскости, вычисление вектора до источника света и углов между векторами.